

Análisis y Diseño con Patrones

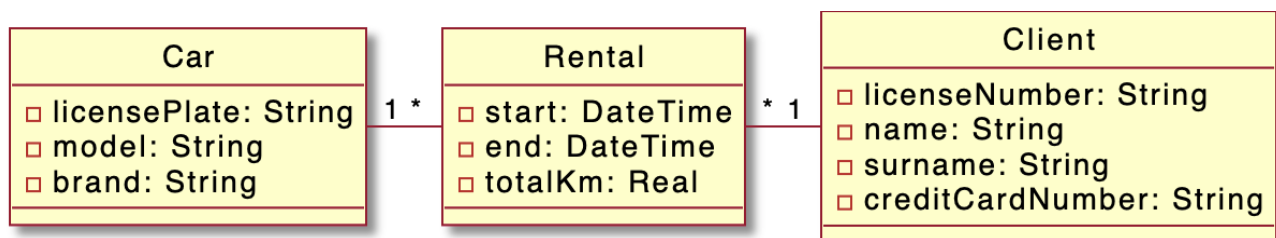
PEC 2: Patrones de Diseño y de Asignación de Responsabilidades

Pregunta 1 (30%)

Enunciado

Una empresa de alquiler de coches nos ha contratado para actualizar su software de gestión. Los clientes se identifican por el número de carné de conducir, y también se almacena en el sistema su nombre, apellidos y número de tarjeta de crédito. Los coches se identifican por la matrícula, y su marca y modelo también se almacenan en el sistema.

Un alquiler incluye una fecha de inicio y finalización así como el número total de kilómetros que el cliente ha recorrido con el coche.



Queremos añadir una nueva funcionalidad para hacer un seguimiento de los daños que ha sufrido el coche durante cada alquiler. Cada desperfecto tiene que incluir una descripción, un precio en euros y si ha sido reparado o no.

En cuanto a los daños al coche, queremos añadir las siguientes operaciones:

- Añadir un nuevo daño a un alquiler de un coche con su descripción y precio. Cuando se añade un daño a un alquiler, este todavía no está reparado.
- Obtener todos los daños de un alquiler.

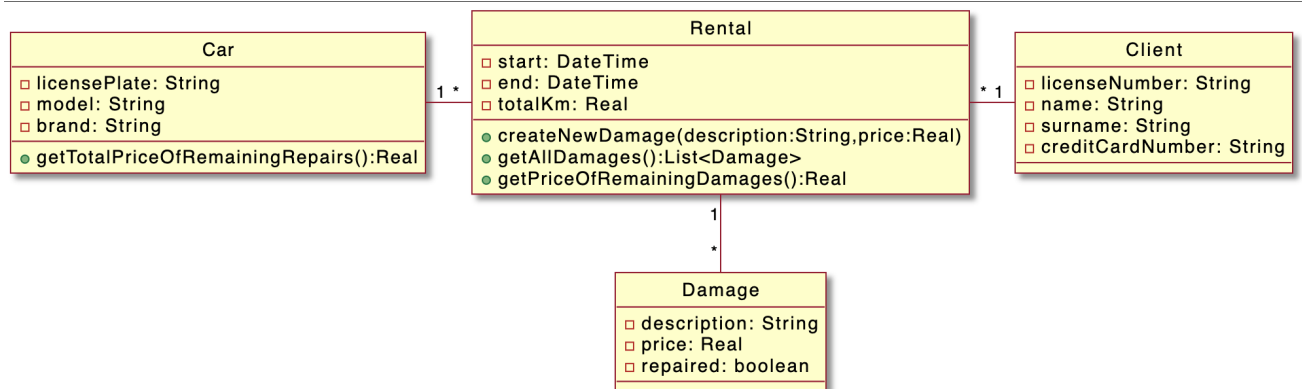
- Obtener el precio total de los daños pendientes de un coche, es decir, los de todos sus alquileres que todavía no se han reparado. Queremos que esta operación tenga el mínimo acoplamiento posible.

Se pide:

- a) Por cada una de las tres operaciones, explica qué patrones de responsabilidad tienes que aplicar.
- b) Modifica el diagrama de clases para incorporar los daños de los alquileres y las tres operaciones descritas.
- c) Proporciona el pseudocódigo de las clases con las operaciones o los diagramas de secuencia para las 3 operaciones.

Solución

- a) Hemos aplicado los siguientes patrones de responsabilidad para cada operación:
 - Añadir un nuevo daño a un alquiler de un coche: El patrón de responsabilidad **Creador** nos permite asignar al método *createNewDamage* de la clase *Rental* la creación de *Damage* ya que *Rental* contiene un conjunto de *Damages*.
 - Obtener todos los daños de un alquiler: El patrón **Experto** se aplica para decidir que la clase *Rental* tiene que definir la operación ya que la clase tiene todos los daños que se han producido durante los periodos de alquiler.
 - Obtener el precio total de los daños pendientes de un coche: En este caso, la clase *Car* es la única que tiene toda la información necesaria para hacer los cálculos. Por lo tanto, aplicamos el patrón **Experto** para asignar la operación en la clase.
- b) Diagrama de clases: El diagrama de clases muestra una nueva clase *Damage* (con sus atributos) y una asociación con *Rental* para representar los daños de los alquileres. Además, las operaciones se han definido en sus clases siguiendo los patrones descritos en la sección anterior.



c) Pseudocódigo:

```

class Rental {
    ...

    public void createNewDamage(String description, Real price) {
        Damage d = new Damage();
        d.description = description;
        d.price = price;
        d.repaired = false;
        this.damages.add(d)
    }

    public List<Damage> getAllDamages() {

        return this.damages;
    }

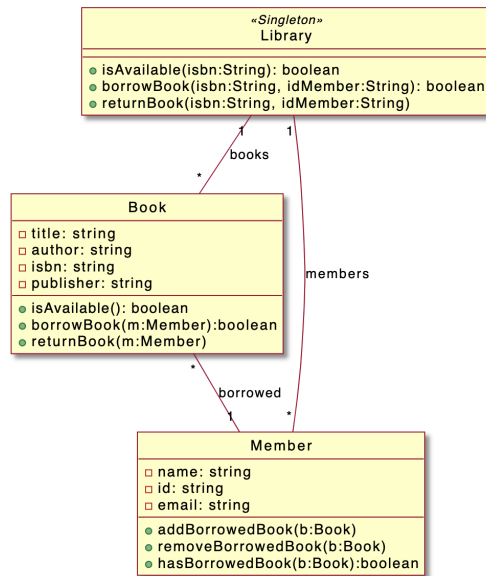
    public Real getPriceOfRemainingDamages() {
        Real result = 0;
        for(Damage d : this.damages) {
            if(!d.repaired) {
                result = result + d.price;
            }
        }
        return result;
    }
}
    
```

```
class Car {  
  
    ...  
  
    public Real getTotalPriceOfRemainingRepairs() {  
        Real result = 0;  
        for(Rental r : rentals) {  
            result = result + r.getPriceOfRemainingDamages();  
        }  
        return result;  
    }  
}
```

Pregunta 2 (30%)

Enunciado

Library System es una aplicación software que gestiona los libros y socios de una biblioteca. La biblioteca dispone de una colección de libros que los socios pueden coger prestados. Cada libro tiene un título, autor, editor e ISBN. Los socios pueden coger libros prestados de la biblioteca. Además, la biblioteca hace un seguimiento de la disponibilidad de cada libro para evitar que los socios puedan coger prestados los libros que no están disponibles. Empezamos a definir el sistema con el siguiente diseño inicial:



En este diseño, la clase **Library** actúa como Fachada y redirige los métodos a las clases adecuadas, tal como se muestra en el pseudocódigo siguiente:

Pseudocódigo

```

class Library {
    Dictionary<String,Book> books;
    Dictionary<String,Member> members;

    public boolean isAvailable(String isbn) {
        Book b = books.get(isbn);

        return b.isAvailable();
    }

    public boolean borrowBook(String isbn, String idMember) {
        Book b = books.get(isbn);
        Member m = members.get(idMember);

        return b.borrowBook(m);
    }

    public void returnBook(String isbn, String idMember) {
        Book b = books.get(isbn);
        Member m = members.get(idMember);

        b.returnBook(m);
    }
}
    
```

```
}
```

Queremos acabar de definir las operaciones anteriores para permitir los préstamos de los libros. Si un libro está disponible, se puede pedir prestado directamente, pero si no está disponible, obviamente, no se podrá prestar.

El método `Library::borrowBook(String ISBN, String idMember)`: boolean devuelve un booleano para indicar si el préstamo se ha producido o no, y hace el préstamo si es posible.

El método `Library::returnBook(String ISBN, String idMember)` devuelve un libro a la biblioteca. Si el socio que devuelve el libro no es el que ha hecho el préstamo anteriormente, se lanzará una excepción. Si el libro no estaba prestado, también se lanzará una excepción.

El método `Library::isAvailable(String ISBN)` devuelve cierto si el libro está disponible.

Los métodos `Member::addBorrowedBook(b:Book)` y `Member::removeBorrowedBook(b:Book)` gestionan la asociación prestada entre `Member` y `Booked`, añadiendo y eliminando los libros que tiene el socio. El método `Member::hasBorrowedBook(b:Book)`: Boolean devuelve cierto si el socio tiene el libro indicado al parámetro.

Se pide:

- Justifica qué patrón de diseño se tiene que aplicar para completar el diseño de estas tres operaciones de la biblioteca.
- Muestra el diagrama de clases con las operaciones necesarias para completar el diseño de estas tres operaciones de la biblioteca.
- Muestra el pseudocódigo completo de las clases con las operaciones o los diagramas de secuencia de las operaciones del diseño propuesto.

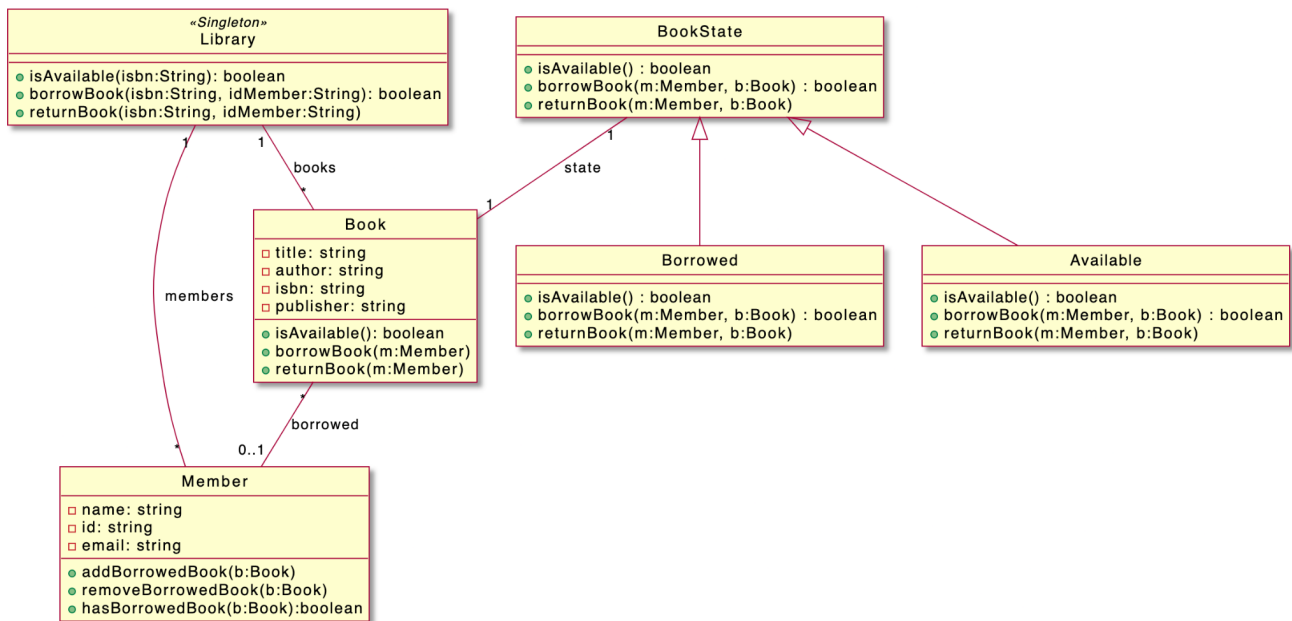
Solución

- El patrón **State** se puede aplicar a esta funcionalidad para representar los diferentes estados por los que puede pasar un libro (prestado o disponible) con clases estado, y cada clase que representa un estado implementará el comportamiento esperado cuando un objeto esté en aquella clase.

En este caso específico, la clase *Book* tiene una nueva asociación con la clase abstracta *BookState*. Cada estado tiene una nueva subclase de *BookState*: *Available* y *Borrowed*.

Nuevos estados pueden ser añadidos fácilmente y la lógica de los préstamos está distribuida por los estados.

b) Diagrama de clases:



c) Pseudocódigo:

```

class Library {
    Dictionary<String,Book> books;
    Dictionary<String,Member> members;

    public boolean isAvailable(String isbn) {
        Book b = books.get(isbn);

        return b.isAvailable();
    }

    public boolean borrowBook(String isbn, String idMember) {
        Book b = books.get(isbn);
        Member m = members.get(idMember);

        return b.borrowBook(m);
    }

    public void returnBook(String isbn, String idMember) {
        Book b = books.get(isbn);
        Member m = members.get(idMember);

        b.returnBook(m);
    }
}
    
```

```

    }
}

class Book {
    String title;
    String author;
    String isbn;
    String publisher;
    BookState state;

    public void setState(BookState state) {
        this.state = state;
    }

    public boolean isAvailable() {
        return state.isAvailable();
    }

    public boolean borrowBook(Member m) {
        return state.borrowBook(this, m);
    }

    public void returnBook(Member m) {
        state.returnBook(this, m);
    }
}

abstract class BookState {
    public abstract boolean isAvailable();
    public abstract boolean borrowBook(Book b, Member m);
    public abstract void returnBook(Book b, Member m);
}

class Available extends BookState {

    public boolean isAvailable() {
        return true;
    }

    public boolean borrowBook(Book b, Member m) {
        b.setState(new Borrowed());
        m.addBorrowedBook(b);
        return true;
    }

    public void returnBook(Book b, Member m) {
        throw new Exception("Book is not borrowed");
    }
}

```



```
class Borrowed extends BookState {
    public boolean isAvailable() {
        return false;
    }

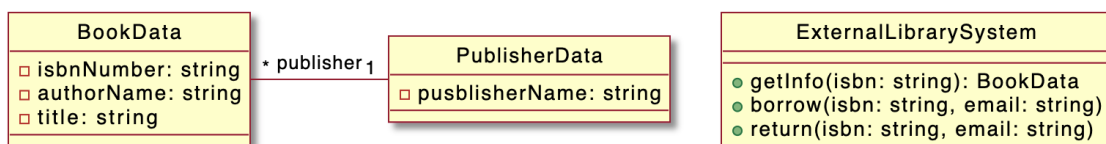
    public boolean borrowBook(Book b, Member m) {
        return false;
    }

    public void returnBook(Book b, Member m) throws Exception {
        if(!m.hasBorrowedBook(b)) {
            throw new Exception("Book is not borrowed by member!");
        } else {
            m.removeBorrowedBook(b);
            b.setState(new Available());
        }
    }
}
```

Pregunta 3 (20%)

Enunciado

Nuestra biblioteca se ha integrado en un consorcio de bibliotecas, por lo cual ahora se puede pedir prestados libros a otras bibliotecas. Nos han proporcionado unas cuantas clases sencillas para integrar los sistemas. La clase External Library System se encarga de almacenar y dar acceso a los datos de los libros de otras bibliotecas mediante la clase BookData. Esta clase es equivalente pero diferente a la clase Book de nuestro sistema.



Como esta integración es bastante compleja, se nos ha pedido que implementemos algún mecanismo de registro (log) para validar que todo funciona correctamente. Se nos pide que registremos cada vez que se llama a los métodos para obtener información, prestar o devolver un libro del sistema de bibliotecas externos. Podéis suponer la existencia de una función *log(mensaje)*:

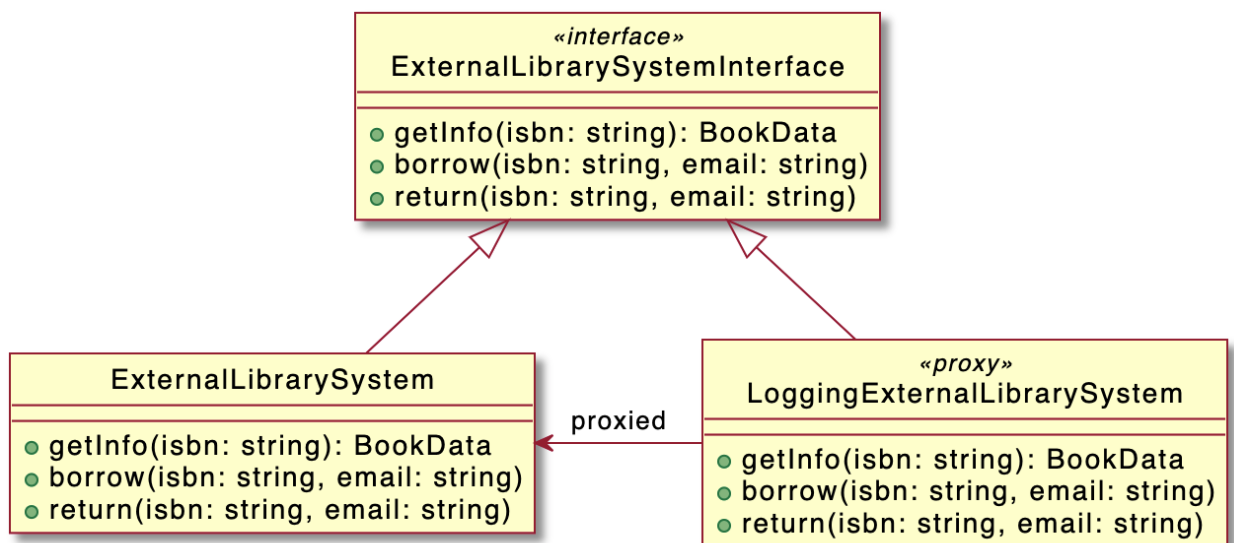
string) para realizar este registro y almacenar en el mensaje la función que se ha llamado con sus parámetros.

Se pide:

- Justifica qué patrón de diseño se tiene que aplicar para incorporar esta funcionalidad.
- Muestra el diagrama de clases con las operaciones necesarias para la nueva funcionalidad.
- Muestra el pseudocódigo de las clases con las operaciones o los diagramas de secuencia para esta funcionalidad.

Solución

- El patrón **Proxy** propone añadir una clase (Proxy) que controlará el acceso al objeto real. En nuestro caso, las llamadas se interceptan en el proxy para escribir al *log* cada vez que una operación se llama.
- Diagrama de clases:



- Pseudocódigo:

```

interface ExternalLibrarySystemInterface {
    BookData getInfo(String isbn);
    borrow(String isbn, String email);
    return(String isbn, String email);
}
    
```

```
class LoggingExternalLibrarySystem {  
  
    private ExternalLibrarySystemInterface proxied;  
  
    LoggingExternalLibrarySystem(ExternalLibrarySystemInterface proxied)  
    {  
        this.proxied = proxied;  
    }  
  
    public BookData getInfo(String isbn) {  
        return proxied.getInfo(isbn);  
    }  
  
    public borrow(String isbn, String email) {  
        log("Calling borrow with isbn "+isbn+" and email "+email);  
        return proxied.borrow(isbn, email);  
    }  
  
    public return(String isbn, String email) {  
        log("Calling return with isbn "+isbn+" and email "+email);  
        return proxied.return(isbn, email);  
    }  
}
```

Pregunta 4 (20%)

Enunciado

Cómo hemos visto en el ejercicio anterior, la clase que utiliza el sistema bibliotecario externo para almacenar la información de los libros (BookData) no es la misma que la que utiliza nuestro sistema (Book). Durante la puesta en marcha de nuestro sistema se nos envía una lista de objetos BookData del consorcio que tenemos que gestionar. Se nos pide que integremos estos libros a nuestro sistema manteniendo la clase BookData puesto que en el futuro la usaremos para sincronizar los préstamos con el sistema de biblioteca externo. De momento, todos los préstamos estarán controlados por nosotros. Tened en cuenta que la clase BookData mantiene básicamente casi la misma información que la clase Book a nuestro diseño, pero con nombres y formatos diferentes.

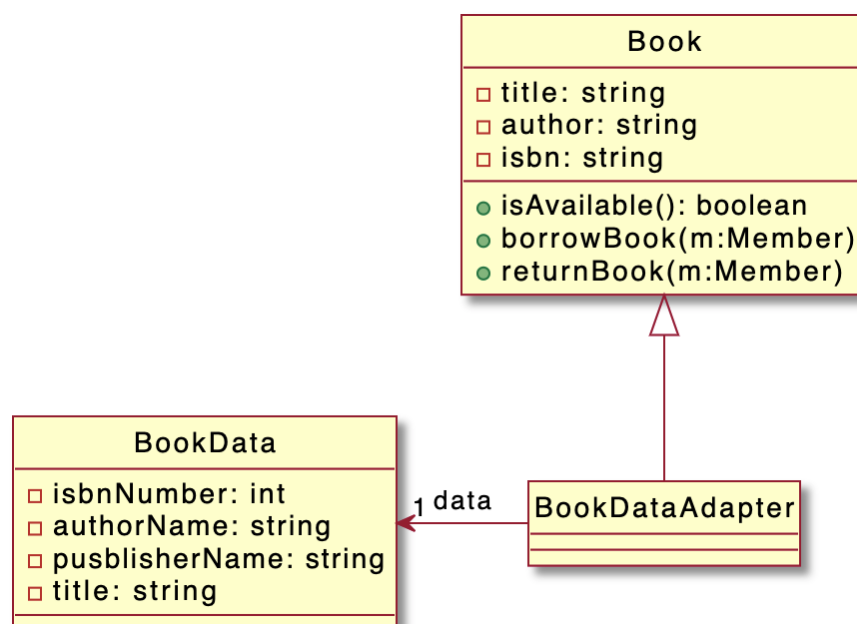
Podéis suponer la existencia del getter/setter y de los constructores con el comportamiento por defecto. Si necesitáis que estas operaciones hagan algo diferente, tenéis que definirlos. Y, como siempre, podéis adaptar el patrón a las necesidades de tu diseño.

Se pide:

- Justifica qué patrón de diseño se tiene que aplicar para incorporar esta nueva funcionalidad.
- Muestra el diagrama de clases con la nueva funcionalidad.
- Muestra el pseudocódigo de las clases con las operaciones o los diagramas de secuencia para esta funcionalidad.

Solución

- El patrón **Adapter** nos permite usar una clase con operaciones diferentes a las propuestas inicialmente por una clase original. En este caso, tenemos dos clases similares a *Book*. Muchos atributos tienen la misma semántica/tipo y sólo requieren un cambio de nombre. Pero, el nombre del *publisher* se registra como una asociación de *BookData* mientras que en nuestro sistema se registra como un string. Usamos getters y setters para proporcionar las conversiones requeridas. Notad que al extender la clase *Book*, *BookDataAdapter* hereda los métodos *isAvailable*, *borrowBook* y *returnBook*.
- Diagrama de clases:



c) Pseudocódigo:

```
class BookDataAdapter extends Book {  
  
    private BookData data;  
  
    public BookDataAdapter(BookData data) {  
        this.data = data;  
    }  
  
    public String getTitle() {  
        return data.getTitle();  
    }  
  
    public String getAuthor() {  
        return data.getAuthorName();  
    }  
  
    public String getIsbn() {  
        return data.getIsbnNumber();  
    }  
  
    public String getPublisher() {  
        return data.publisher.getPublisherName();  
    }  
  
    public void setTitle(String title) {  
        data.setTitle(title);  
    }  
  
    public void setAuthor(String author) {  
        data.setAuthorName(author);  
    }  
  
    public void setIsbn(String isbn) {  
        data.setIsbnNumber(isbn);  
    }  
  
    public void setPublisher(String publisher) {  
        Publisher p = new Publisher()  
        p.setPublisherName(publisher);  
        data.setPublisher(p);  
    }  
}
```